تحليل ومناقشة نتائج خوارزميات الترتيب

Algorithm Performance Analysis & Discussion ($N = 100,000$)

تقرير التحليل فقط

1. تحليل أداء الخوارزميات (Algorithm Analysis)

أ. الخوارزميات ذات التعقيد التربيعي $O(N^2)$

- **Bubble Sort:** أظهرت أداءً سيئاً وزمن تنفيذ مرتفعاً جداً يزداد بشكل كبير مع كبر حجم البيانات في الحالات العشوائية والعكسية والجزئية (وصلت إلى 19.7 ثانية)، بينما سجلت أداءً سريعاً جداً (271 ميكروثانية) في حالة البيانات المرتبة تماماً بفضل خاصية التوقف المبكر عند عدم وجود تبديلات.
- **Selection Sort:** أظهرت أزمنة تنفيذ متقاربة وبطيئة في جميع الحالات (بين 2.6 إلى 5.6 ثانية)، لأن عدد المقارنات يظل ثابتاً $O(N^2)$ بصرف النظر عن حالة الترتيب الأولي للمصفوفة.
- **Insertion Sort:** أظهرت بطئاً في الحالات العشوائية والعكسية، لكنها حققت **أفضل أداء على الإطلاق** في حالة البيانات المرتبة تماماً (143 ميكروثانية فقط) بفضل التعقيد الزمني الخطي $O(N)$ في الحالة الأفضل.

ب. الخوارزميات ذات التعقيد اللوغاريتمي $O(N \log N)$

- **Merge Sort:** حققت كفاءة عالية وأداءً ثابتاً ومستقراً في جميع حالات البيانات (10 إلى 57 مللي ثانية)، نظراً لضمان تعقيدها الزمني $O(N \log N)$ دائماً باستخدام أسلوب "فرق تسد" (Divide and Conquer).
- **Quick Sort:** حققت **أفضل أداء إجمالي وأسرع زمن تنفيذ** في البيانات العشوائية والعكسية والجزئية؛ وذلك بفضل استخدام المحور العشوائي (Randomized Pivot) وتجنب طفح المكدس (StackOverflowError) أو التدهور للحالة الأسوأ $O(N^2)$.

2. الاستنتاج الرئيسي والتوصية (Conclusion)

💡 الخلاصة والتوصيات البرمجية:

- **الفارق الهيكلي:** تُثبت النتائج الفارق الشاسع بين الخوارزميات ذات التعقيد التربيعي $O(N^2)$ والإنتاجية العالية لخوارزميات $O(N \log N)$ عند التعامل مع أحجام بيانات كبيرة ($N=100,000$).
- **أهمية المحور:** يُعد اختيار المحور العشوائي حاسماً لضمان استقرار أداء **Quick Sort** ومنع تدهوره في الحالات المرتبة.
- **التوصية العملية:** تظل **Insertion Sort** الخيار الأمثل والأسلس للمصفوفات المرتبة جزئياً أو كلياً، بينما تعتبر **Quick Sort** و **Merge Sort** الخيار الأفضل والأساسي للبيانات العامة والكبيرة.